

LinkedStack.c

연결 리스트(Linked List)를 이용한 **스택(Stack)** 자료구조의 구현 예제입니다.

개요

이 파일은 `LinkedStackType` 구조체를 사용하여 **동적 메모리 기반의 스택**을 구현하고, `push`, `pop`, `peek`, `print_stack` 등의 기본 연산을 처리합니다.

스택은 **후입선출(Last-In-First-Out, LIFO)** 구조를 따릅니다.

주요 기능

◇ 스택 초기화

```
void init(LinkedStackType *s);
```

- `top` 포인터를 `NULL`로 초기화하여 빈 스택 생성

◇ 스택 상태 확인

```
int is_empty(LinkedStackType *s);
```

- `top == NULL`이면 비어 있음

```
int is_full(LinkedStackType *s);
```

- 연결 리스트 기반 스택은 항상 공간이 존재하므로 `0` 반환

◇ 데이터 삽입 (push)

```
void push(LinkedStackType *s, element item);
```

- 새 노드를 동적으로 생성하여 `top`에 삽입

◇ 데이터 삭제 및 반환 (pop)

```
element pop(LinkedStackType *s);
```

- **top** 노드를 제거하고, 다음 노드를 **top**으로 설정
- 반환된 노드의 **data** 값을 반환
- 비어 있을 경우 에러 메시지 출력 후 종료

◇ 최상단 값 조회 (peek)

```
element peek(LinkedStackType *s);
```

- **top**의 데이터를 확인만 하고 제거하지 않음

◇ 스택 출력

```
void print_stack(LinkedStackType *s);
```

- **top**부터 아래 방향으로 순회 출력
- 출력 예시: 30 -> 20 -> 10 -> NULL

🔗 구조체 정의

```
typedef struct StackNode {
    element data;
    struct StackNode *link;
} StackNode;

typedef struct {
    StackNode *top;
} LinkedStackType;
```

- **StackNode**: 데이터와 다음 노드를 가리키는 포인터
- **LinkedStackType**: 전체 스택을 대표하는 구조체

🔗 컴파일 & 실행

```
# 컴파일
gcc -o LinkedStack LinkedStack.c
```

```
# 실행
./LinkedStack
```

⚠ 한글 오류 메시지가 있는 경우, 출력 인코딩 설정을 확인하세요.

💡 학습 포인트

- ☒ 연결 리스트를 활용한 스택 구현
- ☒ 동적 메모리 할당 및 해제 (`malloc`, `free`)
- ☒ LIFO 구조에 대한 명확한 이해
- ☒ 예외 처리 (`is_empty`, `exit()` 등)

🔍 스택 동작 예시

```
Push(10) → Push(20) → Push(30)
Top → [30] → [20] → [10] → NULL

Pop() → returns 30
Top → [20] → [10] → NULL
```